Week 9: A Proportionate Processing Architecture, Chosen from Measured Needs

Built for Cars, Deadly for Walkers: mining the roadway conditions behind Florida's pedestrian deaths, 2020–2024

Yelysei Shkirpan · Capstone Project, B.S. Data Science · Eastern Florida State College · October 18, 2026

Repository: https://github.com/Sor1na/florida-pedestrian-patterns

The decision in one sentence. The project keeps its current architecture — Python (pandas, pandera) feeding a local PostgreSQL 16 database, run on demand, tested on every change by GitHub Actions — because the measured workload is 168 MB of input, 15,697 stored rows, and a 22-second end-to-end run that repeats at most once a year; a distributed platform would cost more time to start than the whole run takes, and it would move person-level rows off the local machine that the proposal's ethics section keeps them on.

Required item	Where
1. Data volume, format variety, update frequency, processing requirements	section 1
2. Measured run-time, memory, storage, query results	section 2 (measured) and 2.3 (estimates, labelled)
3. Comparison of at least two feasible architectures	section 3 (three architectures)
4. Justified selection, including the decision not to use a distributed platform	section 4
5. Partitioning, storage format, security, cost, reproducibility	section 5
6. Conditions that would change the decision	section 6
How the architecture affects the data story	section 7

Every number below comes from a file in this repository. Measured values name the log they come from. Values that have not been measured yet are marked **estimate** and come with the command that measures them.

---

1. What the project processes

1.1 Volume, formats, update frequency

Source	Format	Size	Rows the pipeline reads	Rows it keeps	Changes how often
FARS 2020–2024 (NHTSA)	5 zip files, 34 CSV members each; mixed encodings (three 2020 members cp1252, the rest UTF-8)	167.7 MB zipped (31.0 / 35.2 / 34.7 / 34.2 / 32.7 MB)	4 members per year: 987,013 national rows (accident 189,208; vehicle 291,638; person 461,187; pbtype 44,980)	87,488 Florida rows → 15,362 stored rows: 3,690 crashes, 4,200 vehicles, 3,736 deaths, 3,736 typing records	Once a year: a new data year each December, and one replacement when the provisional 2024 Annual Report File becomes Final
Census county estimates (Vintage 2025)	1 CSV, latin-1, 3,195 rows × 99 columns	2.1 MB	3,195	67 counties → 335 county-years	Once a year (new vintage)
EPA Smart Location Database v3 / National Walkability Index	1 file geodatabase	about 1 GB (**projected**, Week 6 inventory; not yet downloaded)	all U.S. block groups	Florida block groups only, roughly 11–12 thousand (**projected**)	Static: released 2021 on 2019 boundaries

The raw files carry 45 to 203 columns per member; the pipeline keeps 41 (accident 16, vehicle 6, person 10, pbtype 9: gate G2 lines in `evidence/week7/runs/run_0001.log`). The whole stored dataset is 15,697 data rows across five tables, plus bookkeeping rows per run (one `etl_run`, 6 `source_file`, about 100 `etl_log`, 94 `validation_result`). Sources: `profile/01_inventory.csv`, `pipeline/sources.csv`, Week 6 §1, Week 7 §1.

1.2 Processing requirements

- **Load path.** Verify checksums (G1), read and filter 987,013 national rows to Florida (G2), reconcile against the profiled inventory (G3), derive the unit of observation, validate 103,185 rows with pandera (G4), reconcile totals (G5), replace 2020–2024 in one transaction, verify inside it (G6). Week 7 §1.
- **Analysis path (Weeks 10–11).** FP-Growth with mlxtend on about 3,053 transactions × 16 items from `v_ped_transactions`; four-of-five-year stability; a swap-randomization permutation baseline, which reruns the miner some hundreds of times; a 2024 hold-out. These are in-memory computations over a few thousand rows.
- **Geography (Week 10, deferred from Week 9).** One point-in-polygon join of 3,687 geocoded crashes against the Florida block groups, once per load; walkability enters the transactions as one ordinal item.

- **Consumption.** County rates and pattern counts through views; a dashboard role that reads aggregates only; a written report.
- **Users and cadence.** One writer (me), on demand. Readers: the analyst role and the dashboard role. No concurrent writers, no service-level agreement, no streaming input.

---

2. Measurements

2.1 Measured on the real files (Windows PC, PostgreSQL 16.15, commit `e63f5f4`, October 11, 2026)

Source: `evidence/week7/runs/run_0001.log` (first run on an empty database) and `run_0002.log` (rerun: delete and re-insert). Every stage writes a `stage … finished in N s` line, so these numbers are reproduced by `python pipeline/run_scenarios.py` on the pinned files.

Stage	Run 1 (empty DB)	Run 2 (rerun)	What it did
preflight	0.20 s	0.21 s	lock, run row, fingerprint of 5 tables
verify_sources (G1)	0.53 s	0.49 s	SHA-256 of 169.8 MB of input
extract (G2)	**18.89 s**	18.79 s	decode 21 members, parse 987,013 rows, keep 87,488
reconcile_inventory (G3)	0.01 s	0.01 s	22 count checks against the Week 6 inventory
transform	0.56 s	0.52 s	unit of observation, filler coordinates, striking-vehicle links
validate_records (G4)	1.15 s	1.15 s	pandera rules on 103,185 rows, 0 rejected
reconcile_unit (G5)	0.01 s	0.01 s	10 count comparisons
load	0.52 s	0.94 s	COPY of 15,697 rows (run 2: delete 15,697, insert 15,697)
verify_load (G6)	0.08 s	0.10 s	25 post-load queries, including the 7 `v_dq_*` views
finalize	0.05 s	0.06 s	md5 fingerprint over every stored row
whole run	**22.0 s**	**22.4 s**	18:27:48.165 → 18:28:10.164

Three readings of this table drive the decision:

1. **86% of the run is reading files, not computing.** The extract parses the national CSVs and discards 91% of their rows. Everything after it — validation, load, verification — takes 2.4 s.
2. **Every query over the full dataset is in milliseconds.** The 25 post-load checks, which scan all five tables and run the seven data-quality views, take 0.08 s together, about 3 ms per query. Hashing every row of every data table takes 0.05 s.
3. **The rerun costs the same as the first run.** Replace-by-year adds 0.4 s of deletes. There is no incremental path to optimize.

2.2 Measured elsewhere

Measurement	Value	Source
Same pipeline on the synthetic files (16% of the real row volume)	1.4 s	`evidence/week7_synthetic/runs/run_0001.log`
Full CI suite on a GitHub-hosted `ubuntu-24.04` runner: fresh PostgreSQL, build, 56 constraint tests, 6 scenarios, 2 traces	51 s (run #4, Oct 11), 55 s (run #3)	GitHub Actions history of `ci.yml`
Week 6 profile: every member read once, eleven statistics tables written	about 30 s	Week 7 §6.1, `evidence/06_profile_run.log`
Record validation throughput	103,185 rows / 1.15 s ≈ 90,000 rows per second	run 1 log

2.3 Not yet measured: estimates, with the command that replaces them

Quantity	Estimate	Reasoning	How to measure (Week 10 submission)
Database on disk	under 10 MB added by the data tables and indexes; under 25 MB added by a semester of bookkeeping and logs	**measured baseline:** the built schema with seeds and the 16-row fixture, before any load, is 9.1 MB (`evidence/week9/storage_and_queries_fixture.log`; most of it is PostgreSQL's per-database overhead). The load adds 15,697 narrow rows (smallint codes, one text column) at a few hundred bytes each with 28 indexes, and each run adds about 200 bookkeeping rows and a 25 KB log file	`psql -d ped_safety -f tools/measure_storage_and_queries.sql` (added this week): `pg_database_size`, per-table `pg_total_relation_size`

Quantity	Estimate	Reasoning	How to measure (Week 10 submission)
Peak memory of `etl.py run`	0.3–0.6 GB	the largest member read at once is `person.csv` 2021: 97,511 rows × 126 columns as text; pandas keeps only 10 columns but parses the whole file; five years of Florida frames stay in memory until the load	Windows: `Get-Process python | Select PeakWorkingSet64` after the run, or `psutil`; Linux: `/usr/bin/time -v python pipeline/etl.py run` (Maximum resident set size)
Full scan of `v_ped_transactions` (5 joins, 16 computed items, 3,736 rows)	under 50 ms	the 7 `v_dq_*` views, which join the same tables, run in about 3 ms each inside G6; the planner uses sequential scans at this size (`sql/03_indexes.sql`)	`EXPLAIN (ANALYZE, BUFFERS)` statements in `tools/measure_storage_and_queries.sql`
EPA join: read the Florida block groups from a 1 GB geodatabase and join 3,687 points	1–3 minutes, dominated by reading the file	geopandas/pyogrio reads one layer with a filter; an `sjoin` of 3,687 points against about 12,000 polygons with a spatial index is seconds	`etl.py` will log `stage join_block_groups finished in N s` like every other stage

None of these estimates changes the conclusion. If the memory estimate were wrong by a factor of ten, the run would still fit in a laptop's RAM; if the join took ten minutes, it would still run once per load.

---

3. Three feasible architectures

	A. Current: Python + local PostgreSQL	B. Single-file analytics: DuckDB + Parquet	C. Distributed: Spark cluster or cloud warehouse
Engine and placement	pandas/pandera in Python; PostgreSQL 16 on the student's PC; CI on a throwaway PostgreSQL container	DuckDB database file (or Parquet files) next to the code; no server; queries from Python or the DuckDB CLI	PySpark on a cluster (Databricks, EMR) or a cloud warehouse (BigQuery, Snowflake); data in object storage
Load time, measured or expected	**22 s measured**, 19 s of it reading CSV	**expected 5–8 s** (not measured): DuckDB's parallel CSV reader with projection pushdown reads only the 41 needed columns; it reads unzipped files, so one unzip step is added	**expected 1–5 minutes** (not measured): cluster or JVM start-up alone is 10 s to several minutes; the work itself is seconds
Integrity rules the project relies on	PRIMARY KEY, FOREIGN KEY, CHECK, EXCLUDE, deferred "one or more" triggers, advisory lock; 56 constraint tests pass	PRIMARY KEY, FOREIGN KEY, CHECK supported; **no deferred constraints, no EXCLUDE, no triggers**: two of the Week 5 rules move into Python code	**none enforced by the engine**; every gate becomes application code
Security model	three NOLOGIN roles; `ped_dashboard` reads aggregates only; person-level rows never leave the PC; no credentials in the repository	file permissions only; **no roles**; the aggregates-only guarantee becomes an export discipline, not a grant	cloud credentials to manage and rotate; person-level rows stored with a provider; access control is strong but off-machine
Rerun safety, lineage, logs	one transaction, status committed with data, `etl_run` / `etl_log` / `validation_result` / `rejected_record`; proven by six scenarios	transactions exist (single writer); bookkeeping tables would need re-creating; lineage tables easy	jobs are idempotent only by design; lineage via platform tooling; nondeterministic partition order needs care in fingerprints
Set-up and cost	PostgreSQL installer, `pip install`; $0; CI free for a public repository (51 s per run)	`pip install duckdb`; $0; smallest footprint of the three	account, billing, cluster configuration; from a few dollars a month (always-on warehouse) to free tiers with time limits; the largest cost is set-up time
Reproducibility	pinned SHA-256 inputs, expected counts, commit hash in every run, CI rebuild from nothing	same, and a single file is easy to ship; DuckDB versions change the file format	reproducible in principle; depends on provider versions and on a network
Fit for the data story	exact counts, every death a traceable row (`trace.py`), bins live in views	the same, if the Week 5 rules are rewritten in Python	exact counts possible, but the per-row trace from zip line to county rate would cross systems

What is fair to say for B. It is the only alternative that would remove the one measured bottleneck. It would also make the deliverable easier to hand over: one file, no server. If the project were starting today with no schema built, B would be a serious contender.

What is fair to say for C. Nothing in the measured workload needs it. Its benefits — horizontal scaling, managed scheduling, many concurrent users — answer problems this project does not have (section 6 lists the conditions under which it would).

---

4. The selection, and why not a distributed platform

Choice: keep architecture A. The reasons, each tied to evidence:

1. **The workload is small by every measure.** 168 MB of input, 15,697 stored rows, a 22-second run, queries in milliseconds (section 2.1). The slowest stage is a file read that would take 19 s once a year.
2. **The parts that make the data story trustworthy are the parts only A gives without rewriting.** The 56 constraint tests, the deferred "one or more" rules, the EXCLUDE constraint on code lists, and the aggregates-only dashboard role were built and tested in Week 5 and exercised on real data in Week 8. B lacks two of those mechanisms and C lacks all of them.
3. **Every weekly deliverable since Week 5 runs on it,** and CI rebuilds it from nothing in 51 s. Changing the engine now would spend the compressed Weeks 10–11 (Week 8, risk 3) on re-implementation instead of on mining.
4. **The ethics commitment is easier to keep on one machine.** The proposal keeps person-level rows in the local database and publishes aggregates. A is the only architecture where that rule needs no credential, provider, or export discipline.

Why not a distributed platform (C), explicitly. Spark or a cloud warehouse would be justified if the data were too large for one machine's memory or disk, if reruns had to finish faster than one machine can, or if many users needed to query at once. The measured facts are the opposite: the whole input is 168 MB, the largest single member is 97,511 rows, the run is 22 s, the reruns happen once a year, and the readers are two roles on one PC. The fixed overhead of a distributed job — cluster or JVM start-up, serialization, shuffling — is larger than the entire current run. Adding it would make the pipeline slower, costlier, harder to reproduce, and would move real crash records to a third party for no measurable gain. The course text's data-quality concepts (Tan et al., chapter 2) are implemented as gates that run in 2.4 s; they do not need to be parallel.

Why not B now. The gain is about 12–14 s per year (expected, section 3), and the cost is re-implementing two integrity rules in code and giving up database roles. That trade is not worth it this semester. B is kept as the first step to take if the extract ever becomes a problem (section 6), because it can replace the extract stage alone while PostgreSQL stays the system of record.

One change adopted inside A, deferred. If the 19-second extract ever matters (for example, when a CI budget is reached), the cheap fix is pandas' `engine="pyarrow"` reader or a DuckDB read of only the 41 required columns. Expected gain 3–5×; it changes no table, gate, or query. It is not adopted now because 19 s once a year does not justify a code change and a new dependency.

---

5. Considerations

5.1 Partitioning

- **The data is already partitioned by year, logically.** Every FARS file is one year; `sources.csv`, `expected_counts.csv`, gates G3/G5, the load (delete and re-insert per year), the lineage lines, and the release label in `source_file` all work per year. The analysis partitions are the same years: four-of-five-year stability and the 2024 hold-out are filters on `crash.year` in the view.
- **Physical partitioning is rejected.** PostgreSQL declarative partitioning (`PARTITION BY LIST (year)`) would add partition-key rules to every composite foreign key and complicate the EXCLUDE constraint, for tables of 656–919 rows per year. The planner uses sequential scans at this size anyway (`sql/03_indexes.sql`); a partition prune would save nothing measurable.
- **Within the raw inputs,** one zip per year is the unit of integrity (one SHA-256 per file) and of replacement (the 2024 Final file replaces one year, Week 7 §6.5).

5.2 Storage format

- **Raw files stay as downloaded**: zipped CSV, pinned by SHA-256, never unzipped to disk, never edited. The pipeline reads members in memory (`zipfile` → text → pandas). This keeps one copy of the truth and makes G1 meaningful.
- **Stored data keeps FARS codes exactly as published**, in typed `smallint` columns with CHECK lists; labels live in `code_lookup` with year ranges. All binning happens in `v_ped_transactions`, so a changed bin edge never requires a reload. This is a storage-format decision that directly affects the data story (section 7).
- **Analysis extracts** (the transaction table for mlxtend, the permutation results) will be written as Parquet or CSV in `data/interim/`, ignored by Git. They are derived and re-creatable from the view.
- **Evidence** is plain text and CSV, so a reader can diff two runs with no tool.
- **Geography:** only Florida block-group attributes are stored (`block_group` has no geometry column, a Week 5 decision). The join runs in geopandas once per load and stores the 12-digit `geoid10`. **PostGIS is rejected:** one join per year of 3,687 points does not justify an extension, and the dashboard shows counties, not polygons.

5.3 Security

- No real FARS row, password, token, or key is in the repository (`.gitignore` covers raw data, dumps, `.env`). CI uses a throwaway PostgreSQL container with trust authentication and synthetic inputs only.
- Three database roles separate writing (`ped_etl`), analysis (`ped_analyst`), and the dashboard (`ped_dashboard`, aggregates only through `v_dash_pattern_county`; constraint tests E1–E6).

- The data dictionary marks case-level columns (`st_case`, coordinates, `rejected_record.raw_record`) as Restricted; published evidence from real files is masked (`--redact-keys`, Week 7 §6.4).
- FARS is public data, but the proposal's ethics section goes further than the law requires: person-level rows stay on one machine, and only aggregates are published. Architecture A is the one where this costs nothing.
- **Backup** is not a database dump: the database is rebuilt from the pinned inputs and the repository in 22 s. The things to back up are the six raw files (170 MB) and the Git history.

5.4 Cost

- Software: $0 (PostgreSQL, Python, pandas, pandera, psycopg, mlxtend, geopandas; GitHub Actions is free for a public repository, and the suite uses 51 s per run).
- Hardware: the student's PC. No new purchase, no subscription.
- Alternatives: B is also $0; C ranges from free tiers with time limits to recurring charges, plus the time to set up and secure an account. For a one-semester, one-person project, the dominant cost is my time, and C spends the most of it.

5.5 Reproducibility

Already in place: SHA-256 pins for every input; expected counts that the gates enforce; a deterministic synthetic generator so the public CI can run the whole pipeline without real rows; the git commit and the PostgreSQL version in every run log; one command to rebuild the database from nothing; CI on every pull request and push.

Added this week: `tools/measure_storage_and_queries.sql`, the script that turns the estimates of section 2.3 into measurements on any PostgreSQL that holds the project database. It was run once on an empty built schema to prove it works (`evidence/week9/storage_and_queries_fixture.log`); the real-data run is one command on the PC that holds the database.

Gap, to close in Week 10: `requirements.txt` pins lower bounds only. A `requirements.lock` from `pip freeze` on the machine that produced the real-data evidence will be added, so a reader can install the exact versions.

---

6. Conditions that would change the decision

Condition	Threshold at which it matters	What changes
Scope grows from Florida to all states	—	**Nothing.** The extract already reads the national files (19 s); the stored data grows about elevenfold, to roughly 170,000 rows, still tiny for PostgreSQL. This is the most likely scope change and it is free.
More input than one machine reads comfortably: for example adding CRSS (a sample of all police-reported crashes, far larger than FARS), all persons rather than pedestrian deaths, or all years since 1975	a run longer than 15 minutes, or peak memory above half the PC's RAM, or raw input above about 2 GB	First step: replace the extract stage with a column-pruning reader (pyarrow or DuckDB, architecture B's reader) while PostgreSQL stays the system of record. Only beyond tens of gigabytes or hours per run would a cluster (C) be the proportionate answer.
More than one writer, or a hand-off to an agency team	the day a second person needs to run or edit the load	Managed PostgreSQL (same schema and scripts); the NOLOGIN roles become real logins; backups become dumps.
A public dashboard	when planners outside the PC need it	A static export of `v_dash_pattern_county` (a few thousand aggregate rows) to Tableau Public or a Plotly page. Never the person-level database, and never a reason for C.
Frequent updates	more often than quarterly (for example a monthly state crash feed)	A scheduler (Task Scheduler or Prefect) and an incremental load path. The sources change once a year today, so none exists (Week 7 §9).
Interactive or repeated spatial queries, or national block-group joins	when geography is queried, not joined once	PostGIS, with geometry stored.
The EPA join is heavier than projected	longer than about 5 minutes or more memory than the PC has	Read the geodatabase layer in chunks, or hand the join to PostGIS. Decision point: the first measured `join_block_groups` time in Week 10.
CI grows past its budget	the 20-minute timeout, or the free minutes	Split the suite into jobs; cache the synthetic inputs.

---

7. How the architecture shapes the data story

The story the project tells planners is a set of counts with a provenance: *under these conditions, this many people died, in these counties*. The architecture decides whether those counts can be trusted and traced.

- **Every death is a stored row, with its codes as FARS published them.** No sampling, no approximation, no aggregation before storage. The 3,736 deaths in the story are the 3,736 rows in `pedestrian`, and `trace.py` can follow any one of them from its line in the zip, through each cleaning rule, to the view and the county rate (Week 8 §6). A distributed pipeline would keep the counts but not that single-machine, single-command trace.
- **Unknowns stay in the story.** Because the gates never drop a row and the view turns every unknown code into an explicit `unknown` item, the story can say "speed limit unknown for 5.6% of striking vehicles" rather than quietly losing those deaths from a denominator.
- **Bins live in views, not in tables.** When the mining step shows that a speed-limit bin edge should move, the change is one line of SQL and no reload; the stored data, its checksums, and its evidence are untouched. The story can be revised without the data becoming unreproducible.
- **The provisional year is visible.** Replace-by-year and the release label let the story report 2024 as provisional now and swap in the Final file later with the same command, which is exactly how the stability and hold-out results will be labelled (Week 6, issue 2).
- **One source of numbers.** The report, the dashboard, and the mining notebook all read the same views on the same database. There is no second copy that can drift, which is the usual way a data story and its evidence come apart.
- **The dashboard role limits what the story can show, on purpose.** Aggregates only: counts per county and pattern, never a person or a point. The architecture enforces the ethics section instead of relying on the author to remember it.

Completion criteria

Criterion	Where it is shown
The decision is supported by project evidence	Section 2.1: per-stage times from the real-data run of October 11, cited by log file; section 1 volumes from the Week 6 profile; section 4 ties each reason to a measurement
Alternatives and tradeoffs are compared fairly	Section 3: the same eight criteria for all three; B's advantage on the only bottleneck and C's advantages are stated; expected values are labelled as not measured
Complexity is proportionate to the need	Section 4: a 22-second yearly run stays on one machine; section 5.1 and 5.2 reject partitioning and PostGIS with measured reasons; one improvement is named and deliberately deferred
Performance claims are reproducible	Measured values name their log and the command that regenerates them (`run_scenarios.py`); estimates name the measuring command (`tools/measure_storage_and_queries.sql`, `/usr/bin/time -v`) and are to be replaced in Week 10
The memo explains how architecture affects the data story	Section 7

Sources and AI assistance

- Tan, P.-N., Steinbach, M., Karpatne, A., & Kumar, V. (2019). *Introduction to data mining* (2nd ed.). Pearson. Chapter 2 (data quality), chapters 5–6 (association analysis).
- National Highway Traffic Safety Administration. (2026). *FARS National CSV files 2020–2024*. https://static.nhtsa.gov/nhtsa/downloads/FARS/
- U.S. Census Bureau. (2026). *County population totals 2020–2025, Vintage 2025*. https://www.census.gov/data/tables/time-series/demo/popest/2020s-counties-total.html
- U.S. Environmental Protection Agency. (2021). *Smart Location Database v3 / National Walkability Index*. https://www.epa.gov/smartgrowth/smart-location-mapping
- PostgreSQL 16 documentation (table partitioning, constraints, roles); DuckDB documentation (constraints, CSV reader); Apache Spark documentation (cluster mode overview).
- This repository: `docs/evidence/week7/` (real-data run logs), `docs/profile/` (Week 6 measurements), `docs/Week5–8` documents.

I used Claude (Anthropic), an AI assistant, to help assemble this memo from the repository's logs and documents, to draft the comparison table, and to write the measurement script. The measurements come from my own run of the pipeline on the real files on October 11, 2026 (Week 8) and from the CI history; the estimates are labelled as such and will be replaced by measured values. The decision, the thresholds in section 6, and the project's rules come from my approved proposal and my Week 4–8 work. I reviewed the generated material and I am responsible for the content.